JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util.concurrent

Class ThreadLocalRandom

java.lang.Object
    java.util.Random
        java.util.concurrent.ThreadLocalRandom

All Implemented Interfaces:

Serializable

```
public class ThreadLocalRandom
extends Random
```

A random number generator isolated to the current thread. Like the global `Random` generator used by the `Math` class, a `ThreadLocalRandom` is initialized with an internally generated seed that may not otherwise be modified. When applicable, use of `ThreadLocalRandom` rather than shared `Random` objects in concurrent programs will typically encounter much less overhead and contention. Use of `ThreadLocalRandom` is particularly appropriate when multiple tasks (for example, each a `ForkJoinTask`) use random numbers in parallel in thread pools.

Usages of this class should typically be of the form: `ThreadLocalRandom.current().nextX(...)` (where `X` is `Int`, `Long`, etc). When all usages are of this form, it is never possible to accidently share a `ThreadLocalRandom` across multiple threads.

This class also provides additional commonly used bounded random generation methods.

Instances of `ThreadLocalRandom` are not cryptographically secure. Consider instead using `SecureRandom` in security-sensitive applications. Additionally, default-constructed instances do not use a cryptographically random seed unless the system property `java.util.secureRandomSeed` is set to `true`.

Since:

1.7

See Also:

Serialized Form

Method Summary

Concrete Methods

Modifier and Type	Method	Description
static ThreadLocalRandom	current()	Returns the current th ThreadLocalRandom.
DoubleStream	doubles()	Returns an effectively stream of pseudorando values, each between z (inclusive) and one (ex
DoubleStream	doubles(double randomNumberOrigin, double randomNumberBound)	Returns an effectively stream of pseudorando values, each conformin given origin (inclusive) (exclusive).
DoubleStream	doubles(long streamSize)	Returns a stream prod given streamSize num pseudorandom double each between zero (inc one (exclusive).
DoubleStream	doubles(long streamSize, double randomNumberOrigin, double randomNumberBound)	Returns a stream prod given streamSize num pseudorandom double each conforming to the origin (inclusive) and b (exclusive).
IntStream	ints()	Returns an effectively stream of pseudorando values.
IntStream	ints(int randomNumberOrigin, int randomNumberBound)	Returns an effectively stream of pseudorando values, each conformin given origin (inclusive) (exclusive).
IntStream	ints(long streamSize)	Returns a stream prod given streamSize num pseudorandom int val

IntStream	ints(long streamSize, int randomNumberOrigin, int randomNumberBound)	Returns a stream prod given streamSize num pseudorandom int val conforming to the give (inclusive) and bound (
LongStream	longs()	Returns an effectively stream of pseudorando values.
LongStream	longs(long streamSize)	Returns a stream prod given streamSize num pseudorandom long va
LongStream	longs(long randomNumberOrigin, long randomNumberBound)	Returns an effectively stream of pseudorando values, each conformin given origin (inclusive) (exclusive).
LongStream	longs(long streamSize, long randomNumberOrigin, long randomNumberBound)	Returns a stream prod given streamSize num pseudorandom long, e conforming to the give (inclusive) and bound (
protected int	next(int bits)	Generates the next pse number.
boolean	nextBoolean()	Returns a pseudorando value.
double	nextDouble()	Returns a pseudorando value between zero (in one (exclusive).
double	nextDouble(double bound)	Returns a pseudorando value between 0.0 (inc the specified bound (ex
double	nextDouble(double origin, double bound)	Returns a pseudorando value between the spec (inclusive) and bound (
float	nextFloat()	Returns a pseudorando value between zero (in one (exclusive).
double	nextGaussian()	

		Returns the next pseud Gaussian ("normally") double value with mea standard deviation 1.0 random number genera sequence.
int	**nextInt**()	Returns a pseudorando value.
int	**nextInt**(int bound)	Returns a pseudorando value between zero (in the specified bound (ex
int	**nextInt**(int origin, int bound)	Returns a pseudorando value between the spec (inclusive) and the spe (exclusive).
long	**nextLong**()	Returns a pseudorando value.
long	**nextLong**(long bound)	Returns a pseudorando value between zero (in the specified bound (ex
long	**nextLong**(long origin, long bound)	Returns a pseudorando value between the spec (inclusive) and the spe (exclusive).
void	**setSeed**(long seed)	Throws UnsupportedOperatio

Methods inherited from class java.util.Random

nextBytes

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Method Detail

current

`public static ThreadLocalRandom current()`

Returns the current thread's `ThreadLocalRandom`.

Returns:

`the current thread's ThreadLocalRandom`

setSeed

`public void setSeed(long seed)`

Throws `UnsupportedOperationException`. Setting seeds in this generator is not supported.

Overrides:

`setSeed` in class `Random`

Parameters:

`seed - the initial seed`

Throws:

`UnsupportedOperationException - always`

next

`protected int next(int bits)`

Description copied from class: `Random`

Generates the next pseudorandom number. Subclasses should override this, as this is used by all other methods.

The general contract of `next` is that it returns an `int` value and if the argument `bits` is between `1` and `32` (inclusive), then that many low-order bits of the returned value will be (approximately) independently chosen bit values, each of which is (approximately) equally likely to be `0` or `1`. The method `next` is implemented by class `Random` by atomically updating the seed to

```
(seed * 0x5DEECE66DL + 0xBL) & ((1L << 48) - 1)
```

and returning

```
(int)(seed >>> (48 - bits)).
```

This is a linear congruential pseudorandom number generator, as defined by D. H. Lehmer and described by Donald E. Knuth in *The Art of Computer Programming,* Volume 3: *Seminumerical Algorithms*, section 3.2.1.

Overrides:

`next` in class `Random`

Parameters:

`bits - random bits`

Returns:

`the next pseudorandom value from this random number generator's sequence`

nextInt

`public int nextInt()`

Returns a pseudorandom `int` value.

Overrides:

`nextInt` in class `Random`

Returns:

`a pseudorandom int value`

nextInt

`public int nextInt(int bound)`

Returns a pseudorandom `int` value between zero (inclusive) and the specified bound (exclusive).

Overrides:

`nextInt` in class `Random`

Parameters:

`bound - the upper bound (exclusive). Must be positive.`

Returns:

`a pseudorandom int value between zero (inclusive) and the bound (exclusive)`

Throws:

`IllegalArgumentException - if bound is not positive`

nextInt

```
public int nextInt(int origin,
                   int bound)
```

Returns a pseudorandom `int` value between the specified origin (inclusive) and the specified bound (exclusive).

Parameters:

`origin - the least value returned`

`bound - the upper bound (exclusive)`

Returns:

`a pseudorandom int value between the origin (inclusive) and the bound (exclusive)`

Throws:

`IllegalArgumentException - if origin is greater than or equal to bound`

nextLong

```
public long nextLong()
```

Returns a pseudorandom `long` value.

Overrides:

`nextLong in class Random`

Returns:

`a pseudorandom long value`

nextLong

```
public long nextLong(long bound)
```

Returns a pseudorandom `long` value between zero (inclusive) and the specified bound (exclusive).

Parameters:

`bound - the upper bound (exclusive). Must be positive.`

Returns:

`a pseudorandom long value between zero (inclusive) and the bound (exclusive)`

Throws:

`IllegalArgumentException - if bound is not positive`

nextLong

```
public long nextLong(long origin,
                     long bound)
```

Returns a pseudorandom `long` value between the specified origin (inclusive) and the specified bound (exclusive).

Parameters:

`origin - the least value returned`

`bound - the upper bound (exclusive)`

Returns:

`a pseudorandom long value between the origin (inclusive) and the bound (exclusive)`

Throws:

`IllegalArgumentException - if origin is greater than or equal to bound`

nextDouble

```
public double nextDouble()
```

Returns a pseudorandom `double` value between zero (inclusive) and one (exclusive).

Overrides:

`nextDouble in class Random`

Returns:

a pseudorandom double value between zero (inclusive) and one (exclusive)

See Also:

Math.random()

nextDouble

```
public double nextDouble(double bound)
```

Returns a pseudorandom double value between 0.0 (inclusive) and the specified bound (exclusive).

Parameters:

bound - the upper bound (exclusive). Must be positive.

Returns:

a pseudorandom double value between zero (inclusive) and the bound (exclusive)

Throws:

IllegalArgumentException - if bound is not positive

nextDouble

```
public double nextDouble(double origin,
                         double bound)
```

Returns a pseudorandom double value between the specified origin (inclusive) and bound (exclusive).

Parameters:

origin - the least value returned

bound - the upper bound (exclusive)

Returns:

a pseudorandom double value between the origin (inclusive) and the bound (exclusive)

Throws:

IllegalArgumentException - if origin is greater than or equal to bound

nextBoolean

```
public boolean nextBoolean()
```

Returns a pseudorandom boolean value.

Overrides:

`nextBoolean` in class `Random`

Returns:

a pseudorandom boolean value

nextFloat

```
public float nextFloat()
```

Returns a pseudorandom `float` value between zero (inclusive) and one (exclusive).

Overrides:

`nextFloat` in class `Random`

Returns:

a pseudorandom float value between zero (inclusive) and one (exclusive)

nextGaussian

```
public double nextGaussian()
```

Description copied from class: `Random`

Returns the next pseudorandom, Gaussian ("normally") distributed `double` value with mean `0.0` and standard deviation `1.0` from this random number generator's sequence.

The general contract of `nextGaussian` is that one `double` value, chosen from (approximately) the usual normal distribution with mean `0.0` and standard deviation `1.0`, is pseudorandomly generated and returned.

The method `nextGaussian` is implemented by class `Random` as if by a threadsafe version of the following:

```
private double nextNextGaussian;
private boolean haveNextNextGaussian = false;

public double nextGaussian() {
  if (haveNextNextGaussian) {
    haveNextNextGaussian = false;
    return nextNextGaussian;
  } else {
    double v1, v2, s;
    do {
      v1 = 2 * nextDouble() - 1;   // between -1.0 and 1.0
```

```
      v2 = 2 * nextDouble() - 1;   // between -1.0 and 1.0
      s = v1 * v1 + v2 * v2;
    } while (s >= 1 || s == 0);
    double multiplier = StrictMath.sqrt(-2 * StrictMath.log(s)/s);
    nextNextGaussian = v2 * multiplier;
    haveNextNextGaussian = true;
    return v1 * multiplier;
  }
}
```

This uses the *polar method* of G. E. P. Box, M. E. Muller, and G. Marsaglia, as described by Donald E. Knuth in *The Art of Computer Programming*, Volume 3: *Seminumerical Algorithms*, section 3.4.1, subsection C, algorithm P. Note that it generates two independent values at the cost of only one call to `StrictMath.log` and one call to `StrictMath.sqrt`.

Overrides:

`nextGaussian` in class `Random`

Returns:

the next pseudorandom, Gaussian ("normally") distributed double value with mean 0.0 and standard deviation 1.0 from this random number generator's sequence

ints

```
public IntStream ints(long streamSize)
```

Returns a stream producing the given `streamSize` number of pseudorandom `int` values.

Overrides:

`ints` in class `Random`

Parameters:

`streamSize` - the number of values to generate

Returns:

a stream of pseudorandom int values

Throws:

`IllegalArgumentException` - if streamSize is less than zero

Since:

1.8

ints

```
public IntStream ints()
```

Returns an effectively unlimited stream of pseudorandom `int` values.

Overrides:

`ints` in class `Random`

Implementation Note:

This method is implemented to be equivalent to `ints(Long.MAX_VALUE)`.

Returns:

a stream of pseudorandom int values

Since:

1.8

ints

```
public IntStream ints(long streamSize,
                      int randomNumberOrigin,
                      int randomNumberBound)
```

Returns a stream producing the given `streamSize` number of pseudorandom `int` values, each conforming to the given origin (inclusive) and bound (exclusive).

Overrides:

`ints` in class `Random`

Parameters:

`streamSize` - the number of values to generate

`randomNumberOrigin` - the origin (inclusive) of each random value

`randomNumberBound` - the bound (exclusive) of each random value

Returns:

a stream of pseudorandom int values, each with the given origin (inclusive) and bound (exclusive)

Throws:

`IllegalArgumentException` - if streamSize is less than zero, or randomNumberOrigin is greater than or equal to randomNumberBound

Since:

1.8

ints

```
public IntStream ints(int randomNumberOrigin,
                      int randomNumberBound)
```

Returns an effectively unlimited stream of pseudorandom `int` values, each conforming to the given origin (inclusive) and bound (exclusive).

Overrides:

`ints` in class `Random`

Implementation Note:

This method is implemented to be equivalent to `ints(Long.MAX_VALUE, randomNumberOrigin, randomNumberBound)`.

Parameters:

`randomNumberOrigin` - the origin (inclusive) of each random value

`randomNumberBound` - the bound (exclusive) of each random value

Returns:

a stream of pseudorandom int values, each with the given origin (inclusive) and bound (exclusive)

Throws:

`IllegalArgumentException` - if randomNumberOrigin is greater than or equal to randomNumberBound

Since:

1.8

longs

```
public LongStream longs(long streamSize)
```

Returns a stream producing the given `streamSize` number of pseudorandom `long` values.

Overrides:

`longs` in class `Random`

Parameters:

`streamSize` - the number of values to generate

Returns:

a stream of pseudorandom long values

Throws:

`IllegalArgumentException` - if streamSize is less than zero

Since:

1.8

longs

```
public LongStream longs()
```

Returns an effectively unlimited stream of pseudorandom `long` values.

Overrides:

`longs` in class `Random`

Implementation Note:

This method is implemented to be equivalent to `longs(Long.MAX_VALUE)`.

Returns:

a stream of pseudorandom long values

Since:

1.8

longs

```
public LongStream longs(long streamSize,
                        long randomNumberOrigin,
                        long randomNumberBound)
```

Returns a stream producing the given `streamSize` number of pseudorandom `long`, each conforming to the given origin (inclusive) and bound (exclusive).

Overrides:

longs in class Random

Parameters:

streamSize - the number of values to generate

randomNumberOrigin - the origin (inclusive) of each random value

randomNumberBound - the bound (exclusive) of each random value

Returns:

a stream of pseudorandom long values, each with the given origin (inclusive) and bound (exclusive)

Throws:

IllegalArgumentException - if streamSize is less than zero, or randomNumberOrigin is greater than or equal to randomNumberBound

Since:

1.8

longs

```
public LongStream longs(long randomNumberOrigin,
                        long randomNumberBound)
```

Returns an effectively unlimited stream of pseudorandom long values, each conforming to the given origin (inclusive) and bound (exclusive).

Overrides:

longs in class Random

Implementation Note:

This method is implemented to be equivalent to longs(Long.MAX_VALUE, randomNumberOrigin, randomNumberBound).

Parameters:

randomNumberOrigin - the origin (inclusive) of each random value

randomNumberBound - the bound (exclusive) of each random value

Returns:

a stream of pseudorandom long values, each with the given origin (inclusive) and bound (exclusive)

Throws:

IllegalArgumentException - if randomNumberOrigin is greater than or equal to randomNumberBound

Since:

1.8

doubles

```
public DoubleStream doubles(long streamSize)
```

Returns a stream producing the given `streamSize` number of pseudorandom `double` values, each between zero (inclusive) and one (exclusive).

Overrides:

`doubles` in class `Random`

Parameters:

`streamSize` - the number of values to generate

Returns:

a stream of double values

Throws:

`IllegalArgumentException` - if streamSize is less than zero

Since:

1.8

doubles

```
public DoubleStream doubles()
```

Returns an effectively unlimited stream of pseudorandom `double` values, each between zero (inclusive) and one (exclusive).

Overrides:

`doubles` in class `Random`

Implementation Note:

This method is implemented to be equivalent to `doubles(Long.MAX_VALUE)`.

Returns:

a stream of pseudorandom double values

Since:

1.8

doubles

```
public DoubleStream doubles(long streamSize,
                            double randomNumberOrigin,
                            double randomNumberBound)
```

Returns a stream producing the given `streamSize` number of pseudorandom `double` values, each conforming to the given origin (inclusive) and bound (exclusive).

Overrides:

`doubles` in class `Random`

Parameters:

`streamSize` - the number of values to generate

`randomNumberOrigin` - the origin (inclusive) of each random value

`randomNumberBound` - the bound (exclusive) of each random value

Returns:

a stream of pseudorandom double values, each with the given origin (inclusive) and bound (exclusive)

Throws:

`IllegalArgumentException` - if streamSize is less than zero

`IllegalArgumentException` - if randomNumberOrigin is greater than or equal to randomNumberBound

Since:

1.8

doubles

```
public DoubleStream doubles(double randomNumberOrigin,
                            double randomNumberBound)
```

Returns an effectively unlimited stream of pseudorandom `double` values, each conforming to the given origin (inclusive) and bound (exclusive).

Overrides:

`doubles` in class `Random`

Implementation Note:

This method is implemented to be equivalent to `doubles(Long.MAX_VALUE, randomNumberOrigin, randomNumberBound)`.

Parameters:

`randomNumberOrigin` - the origin (inclusive) of each random value

`randomNumberBound` - the bound (exclusive) of each random value

Returns:

a stream of pseudorandom double values, each with the given origin (inclusive) and bound (exclusive)

Throws:

`IllegalArgumentException` - if randomNumberOrigin is greater than or equal to randomNumberBound

Since:

1.8

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.